

Relatório Final - EFC1

Padrões e Arquitetura de Software

Repositório: github.com/brunoccruzz/Padroes-Arquitetura-de-Software

Este relatório documenta a refatoração do sistema de pedidos originalmente concentrado em `legacy_system.py`. A versão final utiliza camadas explícitas, abstrações, Strategy, Factory e Observer, mantendo a fachada `LegacyOrderSystem` para compatibilidade com os Golden Master Tests.

1. Visão geral da arquitetura final

Camada/arquivo	Responsabilidade
<code>legacy_system.py</code>	Fachada de compatibilidade com a API original; delega para serviços.
<code>src/models</code>	Entidades de domínio: <code>Order</code> , <code>OrderItem</code> , <code>PaymentRecord</code> e enums.
<code>src/repositories</code>	<code>OrderRepository</code> , único ponto de acesso direto ao SQLite.
<code>src/services</code>	Orquestra regras de pedido, pagamento e relatório.
<code>src/interfaces</code>	Contratos ABC para repositório, observadores e serviços externos.
<code>src/strategies</code>	Estratégias de desconto e pagamento.
<code>src/factories</code>	Fábricas de pedidos por tipo de cliente.
<code>src/observers</code>	Observadores de notificação desacoplados do serviço de pedido.

2. Identificação das violações SOLID

Princípio	Trecho/arquivo	Justificativa técnica	Impacto prático
SRP - Single Responsibility Principle	<code>legacy_system.py</code> , classe <code>LegacyOrderSystem</code> ; <code>docs/analise_inicial.md</code> linhas 1- 3	A classe criava tabelas, calculava totais, aplicava descontos, processava pagamentos, alterava status e gerava relatórios.	Mudanças em pagamento ou relatório poderiam quebrar criação de pedido e persistência.
SRP - Single Responsibility Principle	<code>legacy_system.py</code> , método <code>create_order</code> ; <code>docs/analise_inicial.md</code> linhas 2- 3	O método misturava normalização de cliente, cálculo de <code>subtotal/desconto/total</code> e <code>INSERT</code> em SQLite.	Testar desconto exigia banco real; trocar regra de negócio impactava SQL.
OCP - Open/Closed Principle	<code>legacy_system.py</code> , cálculo de desconto; <code>docs/analise_inicial.md</code> linha 4	A regra usava <code>if/elif</code> para <code>VIP</code> e <code>CORPORATE</code> , exigindo edição do método para novo tipo de cliente.	Adicionar <code>PARTNER</code> , <code>BLACK</code> ou desconto por volume ficaria caro e arriscado.
OCP - Open/Closed Principle	<code>legacy_system.py</code> , método <code>pay_order</code> ; <code>docs/analise_inicial.md</code> linhas 5- 6	Os meios <code>CARD</code> , <code>PIX</code> e <code>BOLETO</code> estavam fixos em validação e mapa de prefixo.	Adicionar <code>CRYPTO</code> ou <code>TRANSFER</code> exigiria alterar método central já testado.
LSP - Liskov Substitution Principle	<code>legacy_system.py</code> , método <code>_connect</code> ; <code>docs/analise_inicial.md</code>	Não havia contrato substituível para repositório; a lógica	Trocar SQLite por memória, API ou outro banco exigiria alterar regra de negócio.

	linhas 6- 7	dependia de sqlite3 diretamente.	
LSP - Liskov Substitution Principle	legacy_system.py, pay_order com strings; docs/analise_inicial.md linhas 7- 8	Pagamentos eram strings interpretadas internamente, não objetos substituíveis por contrato comum.	Variações reais de gateway virariam condicionais e poderiam quebrar fluxos existentes.
ISP - Interface Segregation Principle	legacy_system.py, interface pública ampla; docs/analise_inicial.md linhas 8- 9	Um consumidor que só cria pedido dependia também de pagamento, cancelamento e relatório.	Clientes simples carregavam capacidades que não usam, aumentando acoplamento.
ISP - Interface Segregation Principle	legacy_system.py, update_status/cancel_order/generate_report; docs/analise_inicial.md linha 9	Leitura/relatório e comandos de escrita estavam no mesmo objeto público.	Código apenas consultivo recebia acesso acidental para alterar estado.
DIP - Dependency Inversion Principle	legacy_system.py, import sqlite3; docs/analise_inicial.md linhas 10-11	Camada de alto nível dependia de detalhe técnico de persistência.	Testes e evolução ficavam presos a SQLite.
DIP - Dependency Inversion Principle	legacy_system.py, _connect instancia banco; docs/analise_inicial.md linha 11	A dependência era criada internamente, impedindo injeção de fake/repositório alternativo.	Criar teste unitário isolado ou migrar banco exigiria editar a classe.

3. Soluções implementadas

Violação	Solução adotada	Padrão GoF	Classes/interfaces	Antes/depois
SRP 1	Separação em OrderService, PaymentService, ReportService e OrderRepository.	Repository / Service Layer	OrderService, PaymentService, ReportService, OrderRepository	Antes: LegacyOrderSystem fazia tudo. Depois: create_order delega para OrderService; pay_order para PaymentService; generate_report para ReportService.
SRP 2	Persistência isolada no OrderRepository; create_order só orquestra criação e salvamento.	Repository	OrderRepositoryInterface, OrderRepository	Antes: create_order calculava e executava INSERT. Depois: OrderService chama factory e repository.save(order).
OCP 1	Descontos extraídos para DiscountStrategyInterface e resolvido por tipo de cliente.	Strategy	NoDiscountStrategy, VipDiscountStrategy, CorporateDiscountStrategy, FixedDiscountStrategy	Antes: if normalized_type == VIP. Depois: discount_resolver.resolve(customer_type).calculate(subtotal).
OCP 2	Pagamentos extraídos para PaymentStrategyInterface e DefaultPaymentStrategyResolver.	Strategy	CardPaymentStrategy, PixPaymentStrategy, BolettoPaymentStrategy	Antes: mapa fixo reference_prefix. Depois: payment_resolver.resolve(method).execute(order)

				r_id, total).
LSP 1	Serviços dependem de OrderRepositoryInterface, permitindo substituição por fake ou outro banco.	Repository + DIP	OrderRepositoryInterface	Antes: sqlite3.connect dentro da classe de negócio. Depois: OrderService(repository: OrderRepositoryInterface, ...).
LSP 2	Pagamentos implementam mesmo contrato execute(order_id, amount) -> PaymentRecord.	Strategy	PaymentStrategyInterface	Antes: method.upper() e condicionais. Depois: qualquer estratégia que cumpra execute é substituível.
ISP 1	Contratos separados por responsabilidade: repositório, observador e resolvedores de estratégia.	Interface Segregation	OrderRepositoryInterface, NotificationObserverInterface	Antes: interface única LegacyOrderSystem. Depois: serviços recebem apenas contratos necessários.
ISP 2	ReportService depende apenas do método report_summary do repositório.	Service + Repository	ReportService	Antes: cliente de relatório usava objeto com comandos. Depois: relatório fica em serviço separado.
DIP 1	Camada de serviço depende de abstrações, não de SQLite.	Dependency Injection	OrderService, PaymentService, ReportService	Antes: import sqlite3 na classe central. Depois: alto nível conhece OrderRepositoryInterface.
DIP 2	A composição concreta fica na fachada LegacyOrderSystem, nas bordas do sistema.	Facade + Dependency Injection	LegacyOrderSystem	Antes: dependências criadas no mesmo lugar da regra. Depois: fachada monta repositório, estratégias e serviços.

3.1 Trechos de código antes e depois

Exemplo 1 - Desconto

```

ANTES
if normalized_type == "VIP":
    discount_rate = 0.10
elif normalized_type == "CORPORATE":
    discount_rate = 0.15

DEPOIS
class DiscountStrategyInterface(ABC):
    def calculate(self, subtotal: float) -> float: ...
class VipDiscountStrategy(DiscountStrategyInterface):
    def calculate(self, subtotal: float) -> float:
        return round(subtotal * 0.10, 2)

```

Exemplo 2 - Pagamento

ANTES

```
reference_prefix = {"CARD": "CARD", "PIX": "PIX", "BOLETO": "BOL"}[normalized_method]
```

DEPOIS

```
payment = self.payment_resolver.resolve(payment_method).execute(order_id, order.total)
self.repository.add_payment(order_id, payment)
```

Exemplo 3 - DIP no serviço

DEPOIS

```
class PaymentService:
    def __init__(self, repository: OrderRepositoryInterface, payment_resolver:
    PaymentStrategyResolverInterface):
        self.repository = repository
        self.payment_resolver = payment_resolver
```

4. Diagrama UML de Classes - Mermaid

```
classDiagram
    direction TB
    class LegacyOrderSystem
    class OrderService
    class PaymentService
    class ReportService
    class OrderRepositoryInterface
    class OrderRepository
    class PedidoFactoryInterface
    class PedidoFactory
    class OrderFactoryInterface
    class CustomerOrderFactory
    class DiscountStrategyInterface
    class DefaultDiscountStrategyResolver
    class PaymentStrategyInterface
    class DefaultPaymentStrategyResolver
    class NotificationObserverInterface
    class EmailNotificationObserver
    class SmsNotificationObserver
    class ManagerNotificationObserver
    class WhatsAppNotificationObserver
    class Order
    class OrderItem
    class PaymentRecord

    LegacyOrderSystem --> OrderService : usa
    LegacyOrderSystem --> PaymentService : usa
    LegacyOrderSystem --> ReportService : usa
    OrderService --> OrderRepositoryInterface : depende
    OrderService --> PedidoFactoryInterface : depende
    OrderService --> NotificationObserverInterface : depende
    PaymentService --> OrderRepositoryInterface : depende
    PaymentService --> PaymentStrategyResolverInterface : depende
    ReportService --> OrderRepositoryInterface : depende
    OrderRepository ..|> OrderRepositoryInterface
    PedidoFactory ..|> PedidoFactoryInterface
    CustomerOrderFactory ..|> OrderFactoryInterface
    DefaultDiscountStrategyResolver --> DiscountStrategyInterface
    DefaultPaymentStrategyResolver --> PaymentStrategyInterface
    EmailNotificationObserver ..|> NotificationObserverInterface
    SmsNotificationObserver ..|> NotificationObserverInterface
    ManagerNotificationObserver ..|> NotificationObserverInterface
    WhatsAppNotificationObserver ..|> NotificationObserverInterface
    Order *-- OrderItem
    Order *-- PaymentRecord
```

5. Melhorias de Clean Code

Nomenclatura: `create_order`, `pay_order`, `report_summary`, `PaymentStrategyInterface` e `DiscountStrategyInterface` expressam intenção.

Métodos extraídos: `_notify_observers`, `_build_items`, `_required_scalar` e métodos do repositório reduziram métodos

longos. Abstrações criadas: Interfaces ABC para repositório, observadores, fábricas e estratégias.

Duplicação eliminada: Cálculo de desconto e geração de pagamento saíram de condicionais duplicáveis para classes especializadas.

Coesão: Cada classe possui motivo principal de mudança: pedido, pagamento, relatório, persistência, notificação ou regra de desconto.

Testabilidade: Serviços aceitam dependências por construtor, permitindo mocks/fakes sem SQLite.

6. Extensões implementadas

Notificação por WhatsApp

Diff conceitual: Adicionado `src/observers/whatsapp_observer.py`; composição registrada em `legacy_system.py`. Nenhuma regra de pedido precisou ser reescrita.

Padrão aplicado: Observer

```
class WhatsAppNotificationObserver(NotificationObserverInterface):
    def update(self, order: Order) -> None:
        print(f"WhatsApp: pedido {order.id} atualizado")
```

Justificativa: Como `OrderService` notifica uma lista de observadores, a extensão entrou apenas como novo observador.

Desconto fixo

Diff conceitual: Adicionado `FixedDiscountStrategy` em `src/strategies/discount_strategy.py`; registro pode ser feito no resolver.

Padrão aplicado: Strategy

```
class FixedDiscountStrategy(DiscountStrategyInterface):
    def __init__(self, amount: float):
        self._amount = amount
    def calculate(self, subtotal: float) -> float:
        return round(min(self._amount, subtotal), 2)
```

Justificativa: Como desconto é estratégia, a regra nova não altera `OrderService` nem `OrderRepository`.

Pagamento por nova modalidade

Diff conceitual: A arquitetura permite adicionar uma classe como `CryptoPaymentStrategy` e registrá-la no `DefaultPaymentStrategyResolver`.

Padrão aplicado: Strategy

```
class CryptoPaymentStrategy(PaymentStrategyInterface):
    def execute(self, order_id: int, amount: float) -> PaymentRecord:
        return PaymentRecord(method=PaymentMethod.CRYPTO, amount=amount, status="APPROVED",
                               reference=f"CRY-{order_id:06d}", paid_at=datetime.now().strftime("%Y-%m-%d %H:%M:%S"))
```

Justificativa: Como pagamento é resolvido por interface, o fluxo de `PaymentService` não muda.

7. Saída dos testes e métricas

```
$ pytest -v
===== test session starts
=====
platform linux -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0 -- /home/brunocruz/Padroes-Arquitetura-de-Software/.venv/bin/python
cachedir: .pytest_cache
rootdir: /home/brunocruz/Padroes-Arquitetura-de-Software
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 63 items

tests/golden_master/test_legacy_behavior.py::test_create_normal_order PASSED
[ 1%]
tests/golden_master/test_legacy_behavior.py::test_create_vip_order PASSED
[ 3%]
tests/golden_master/test_legacy_behavior.py::test_create_corporate_order PASSED
[ 4%]
tests/golden_master/test_legacy_behavior.py::test_payment_methods[card-CARD-000001] PASSED
[ 6%]
tests/golden_master/test_legacy_behavior.py::test_payment_methods[pix-PIX-000001] PASSED
[ 7%]
tests/golden_master/test_legacy_behavior.py::test_payment_methods[boleto-BOL-000001] PASSED
[ 9%]
tests/golden_master/test_legacy_behavior.py::test_update_status PASSED
[ 11%]
tests/golden_master/test_legacy_behavior.py::test_cancel_order PASSED
[ 12%]
tests/golden_master/test_legacy_behavior.py::test_report_generation PASSED
[ 14%]
tests/golden_master/test_legacy_behavior.py::test_paid_order_cannot_be_cancelled PASSED
[ 15%]
tests/unit/test_crypto_payment_strategy.py::test_implements_payment_strategy_interface PASSED
[ 17%]
tests/unit/test_crypto_payment_strategy.py::test_returns_payment_record_instance PASSED
[ 19%]
tests/unit/test_crypto_payment_strategy.py::test_fee_calculation[100.0-102.0] PASSED
[ 20%]
tests/unit/test_crypto_payment_strategy.py::test_fee_calculation[1000.0-1020.0] PASSED
[ 22%]
tests/unit/test_crypto_payment_strategy.py::test_fee_calculation[99.99-101.99] PASSED
[ 23%]
tests/unit/test_crypto_payment_strategy.py::test_fee_calculation[123.4567-125.93] PASSED
[ 25%]
tests/unit/test_crypto_payment_strategy.py::test_fee_calculation[0.01-0.01] PASSED
[ 26%]
tests/unit/test_crypto_payment_strategy.py::test_fee_calculation[999999.99-1019999.99] PASSED
[ 28%]
tests/unit/test_crypto_payment_strategy.py::test_reference_format PASSED
[ 30%]
tests/unit/test_crypto_payment_strategy.py::test_reference_zero_padded_for_large_id PASSED
[ 31%]
tests/unit/test_crypto_payment_strategy.py::test_status_is_approved PASSED
[ 33%]
tests/unit/test_crypto_payment_strategy.py::test_paid_at_is_non_empty_string PASSED
[ 34%]
tests/unit/test_crypto_payment_strategy.py::test_zero_amount_follows_existing_contract PASSED
[ 36%]
tests/unit/test_crypto_payment_strategy.py::test_negative_amount_follows_existing_contract PASSED
[ 38%]
tests/unit/test_crypto_payment_strategy.py::test_very_large_amount PASSED
[ 39%]
tests/unit/test_crypto_payment_strategy.py::test_result_has_same_structure_as_pix_strategy PASSED
[ 41%]
tests/unit/test_crypto_payment_strategy.py::test_crypto_amount_is_two_percent_above_pix PASSED
[ 42%]
```

tests/unit/test_crypto_payment_strategy.py::test_integration_order_ends_in_paid_status PASSED
[44%]
tests/unit/test_notification_observers.py::test_whatsapp_observer_implements_interface PASSED
[46%]
tests/unit/test_notification_observers.py::test_whatsapp_observer_update_runs_successfully PASSED
[47%]
tests/unit/test_notification_observers.py::test_email_observer_implements_interface PASSED
[49%]
tests/unit/test_notification_observers.py::test_email_observer_update_runs_successfully PASSED
[50%]
tests/unit/test_notification_observers.py::test_sms_observer_implements_interface PASSED
[52%]
tests/unit/test_notification_observers.py::test_sms_observer_update_runs_successfully PASSED
[53%]
tests/unit/test_notification_observers.py::test_manager_observer_implements_interface PASSED
[55%]
tests/unit/test_notification_observers.py::test_manager_observer_update_runs_successfully PASSED
[57%]
tests/unit/test_pedido_factory.py::test_pedido_factory_creates_supported_customer_orders[normal-NORMAL-0.0-275.5] PASSED
[58%]
tests/unit/test_pedido_factory.py::test_pedido_factory_creates_supported_customer_orders[vip-VIP-27.55-247.95] PASSED
[60%]
tests/unit/test_pedido_factory.py::test_pedido_factory_creates_supported_customer_orders[corporate-CORPORATE-41.32-234.18] PASSED
[61%]
tests/unit/test_pedido_factory.py::test_customer_factories_are_liskov_substitutable[order_factory0] PASSED
[63%]
tests/unit/test_pedido_factory.py::test_customer_factories_are_liskov_substitutable[order_factory1] PASSED
[65%]
tests/unit/test_pedido_factory.py::test_customer_factories_are_liskov_substitutable[order_factory2] PASSED
[66%]
tests/unit/test_pedido_factory.py::test_pedido_factory_rejects_unsupported_customer_type PASSED
[68%]
tests/unit/test_volume_discount_strategy.py::test_implements_discount_strategy_interface PASSED
[69%]
tests/unit/test_volume_discount_strategy.py::test_no_discount_below_200 PASSED
[71%]
tests/unit/test_volume_discount_strategy.py::test_no_discount_at_zero PASSED [73%]
tests/unit/test_volume_discount_strategy.py::test_5_percent_at_200 PASSED [74%]
tests/unit/test_volume_discount_strategy.py::test_5_percent_just_below_500 PASSED [76%]
tests/unit/test_volume_discount_strategy.py::test_10_percent_at_500 PASSED [77%]
tests/unit/test_volume_discount_strategy.py::test_10_percent_just_below_1000 PASSED [79%]
tests/unit/test_volume_discount_strategy.py::test_15_percent_at_1000 PASSED [80%]
tests/unit/test_volume_discount_strategy.py::test_15_percent_above_1000 PASSED [82%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[0.0-0.0] PASSED [84%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[100.0-0.0] PASSED [85%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[199.99-0.0] PASSED [87%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[200.0-10.0] PASSED [88%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[300.0-15.0] PASSED [90%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[499.99-25.0] PASSED [92%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[500.0-50.0] PASSED [93%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[750.0-75.0] PASSED [95%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[999.99-100.0] PASSED [96%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[1000.0-150.0] PASSED [98%]
tests/unit/test_volume_discount_strategy.py::test_tiers_parametrized[1500.0-225.0] PASSED

[100%]

===== 63 passed in 0.16s

\$ pytest --cov=. --cov-report=term-missing --cov-report=html

===== test session starts

platform linux -- Python 3.13.5, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/brunocruz/Padroes-Arquitetura-de-Software
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 63 items

tests/golden_master/test_legacy_behavior.py	[15%]
tests/unit/test_crypto_payment_strategy.py	[44%]
tests/unit/test_notification_observers.py	[57%]
tests/unit/test_pedido_factory.py	[68%]
tests/unit/test_volume_discount_strategy.py	[100%]

===== tests coverage

===== coverage: platform linux, python 3.13.5-final-0

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
legacy_system.py	40	1	2	1	95%	65
src/__init__.py	0	0	0	0	100%	
src/factories/__init__.py	2	0	0	0	100%	
src/factories/order_factory.py	56	3	4	1	93%	16, 27, 76
src/interfaces/__init__.py	3	0	0	0	100%	
src/interfaces/notification_observer.py	6	1	0	0	83%	9
src/interfaces/repositories.py	18	5	0	0	72%	9, 13, 17, 21, 25
src/interfaces/services.py	6	6	0	0	0%	1-9
src/models/__init__.py	2	0	0	0	100%	
src/models/order.py	41	0	0	0	100%	
src/observers/__init__.py	5	0	0	0	100%	
src/observers/email_observer.py	5	0	0	0	100%	
src/observers/manager_observer.py	6	0	2	0	100%	
src/observers/sms_observer.py	6	0	2	0	100%	
src/observers/whatsapp_observer.py	5	0	0	0	100%	
src/repositories/__init__.py	2	0	0	0	100%	
src/repositories/order_repository.py	73	4	8	3	91%	82, 103, 113-114
src/services/__init__.py	4	0	0	0	100%	
src/services/order_service.py	38	4	12	4	84%	30, 43, 46, 53
src/services/payment_service.py	18	2	4	2	82%	18, 20
src/services/report_service.py	9	0	0	0	100%	
src/strategies/__init__.py	4	0	0	0	100%	
src/strategies/crypto_payment_strategy.py	9	0	0	0	100%	
src/strategies/discount_strategy.py	32	4	0	0	88%	9, 15, 39, 42
src/strategies/payment_strategy.py	28	3	2	1	87%	10, 16, 59
src/strategies/volume_discount_strategy.py	8	0	4	0	100%	
tests/golden_master/test_legacy_behavior.py	63	0	0	0	100%	
tests/unit/test_crypto_payment_strategy.py	58	0	0	0	100%	
tests/unit/test_notification_observers.py	45	0	0	0	100%	
tests/unit/test_pedido_factory.py	30	0	0	0	100%	
tests/unit/test_volume_discount_strategy.py	27	0	0	0	100%	

TOTAL 649 33 40 12 93%

Coverage HTML written to dir htmlcov

Required test coverage of 80.0% reached. Total coverage: 93.47%

===== 63 passed in 0.80s

\$ ruff check .

All checks passed!

8. Reflexão metacognitiva

8.1 O princípio que ainda não dominei

O princípio que ainda tenho mais dificuldade é o LSP. Eu entendo a ideia geral de que uma implementação deve poder substituir outra sem quebrar o cliente, mas ainda confundo LSP com DIP quando uso interfaces. Neste projeto, ficou mais claro que não basta criar uma interface: as implementações precisam obedecer ao mesmo comportamento esperado. Por exemplo, uma `PaymentStrategy` não pode retornar um objeto incompatível ou exigir parâmetros extras, senão ela quebra a substituição mesmo implementando a classe abstrata.

8.2 Como usei IA neste trabalho

Usei IA para revisar as violações SOLID, sugerir uma separação inicial em camadas e garantir que todos os critérios de aceitação do projeto fossem mapeados na documentação. Os prompts que funcionaram melhor foram específicos, como “identifique duas violações por princípio no `legacy_system.py`” e “proponha refatoração mantendo Golden Master Tests”. Prompts genéricos, como “melhore o código”, falharam porque sugeriam mudanças grandes sem preservar compatibilidade. Uma decisão em que discordei da IA foi evitar transformar tudo em várias microinterfaces pequenas demais; mantive interfaces por papel arquitetural para não deixar o projeto artificialmente complexo.

9. Referências do próprio projeto

`README.md` - descrição de arquitetura, comandos e sprints.

`docs/analise_inicial.md` - violações SOLID originais.

`docs/diagram.md` - diagrama de classes em Mermaid.

`pyproject.toml` - configuração de pytest, coverage, mypy e ruff.

`src/services`, `src/repositories`, `src/strategies`, `src/factories`, `src/observers` - código refatorado.